

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

Autonomous Institute under VTU, Belagavi, Karnataka - 590 018

Yelahanka, Bengaluru, Karnataka - 560 119



Computer Networks (BCS502) CCA Report

On

“Detection of Suspicious Network Traffic using Wireshark Packet Analyser”

BACHELOR OF ENGINEERING

in

INFORMATION SCIENCE AND ENGINEERING

by

Naveen Biradar 1BY23IS132

Pavan CV 1BY23IS144

Pranav Adithya SK 1BY23IS155

Rajendra Prasad CL 1BY23IS171

Under the Guidance of

Dr. Kalai Vani Y S

Associate Professor

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Avalahalli, Yelahanka, Bengaluru, Karnataka -560119

2025-26

Evaluation sheet

Name	USN	Introduction and Setup, Application Selection and Relevance (5)	Packet Capture Process(3)	Packet Analysis and Interpretation(5)	Use of Wireshark Features and Practical Insights and Troubleshooting(5)	Documents and Report	Total
		5	3	5	5	2	20

Content

Page No.

1. Introduction	1 1 2 2 2 - 5
2. Abstract	6 6 - 7 7 - 8
3. Problem Statement	9 9
4. System Requirements	
5. Design and Implementation	
6. Security Analysis	
7. System Testing	
8. Innovation and Application relevance	
9. Conclusion	
10. References	

Introduction

The objective of this report is to demonstrate the functionality and application of **Wireshark**, the world's foremost network protocol analyzer. Wireshark is a powerful tool used for troubleshooting, analysis, software and communications protocol development, and education. It captures live network traffic and displays the data in a highly detailed, human-readable format.

This demonstration specifically focuses on capturing and analyzing **HTTP Web Traffic** to understand the fundamental mechanics of a client-server web session. The power of Wireshark lies in its ability to perform **passive network analysis**, allowing us to inspect the raw data streams without affecting the network performance, providing an unparalleled view of network conversations at the byte level. This capability is essential for any modern network engineer or security professional.

Setup and Relevance:

1. **Wireshark Installation:** The installation of Wireshark (e.g., v4.2.0) was successfully installed on the system, including the necessary packet capture driver (Nmap).
2. **Interface Selection:** The active network interface (e.g., Wi-Fi or Ethernet) connected to the internet was successfully identified and selected.
3. **Capture Initiation:** A live capture was initiated on the selected interface, demonstrating a clear understanding of the tool's core function.

Abstract

This report details the process of network traffic inspection using Wireshark, centered on analyzing a standard HTTP web browsing session. By capturing packets generated during a website request, the report provides a detailed breakdown of the dependency on DNS resolution, the TCP/IP connection establishment (three-way handshake), the subsequent HTTP GET request and 200 OK response, and the connection termination. The analysis confirms the layered structure of network communication and highlights Wireshark's capability to provide essential insights into network performance, resource utilization, and potential security vulnerabilities.

Problem Statement

To successfully capture a typical web browsing interaction and perform a **Detailed Packet Analysis and Interpretation** to:

1. Verify the TCP three-way handshake sequence and connection termination process.
2. Identify the Source IP, Destination IP, and port numbers involved in the HTTP session and its dependency (DNS).
3. Examine the structure of the HTTP GET request and the HTTP 200 OK response packets, providing a layer-by-layer breakdown.
4. Demonstrate the effective use of Wireshark's filtering, stream analysis, and statistical features to troubleshoot and gain practical insights.

System Requirements

Type	Component	Specification
Hardware	Processor	Intel Core i3 or equivalent (or better)
	RAM	4 GB (or better)
Software	Operating System	Windows 10/11 or modern Linux/macOS
	Analyzer Tool	Wireshark (Latest Stable Release)
	Auxiliary Tool	Web Browser (e.g., Chrome, Firefox)

Design and Implementation

5.1 Packet Capture Process

The capture was initiated before accessing a non-secure HTTP website (e.g., <http://example.com>).

1. **Interface Selection:** Selected the primary interface (e.g., WLAN).

2. **Capture Filter:** No initial capture filter was used to ensure the capture of all link-layer traffic, including DNS and ARP. This approach is superior for a complete analysis as it captures the entire process, including the crucial prerequisite step of resolving the domain name.

3. **Action Performed:** A request to a sample HTTP site was executed, and the capture was stopped immediately after the page loaded.

Fulfilment: Successful capture was achieved with a sufficient packet volume, allowing for the analysis of the DNS, TCP, and HTTP protocols in sequence.

5.2 Packet Analysis and Interpretation

The captured data was filtered using a **Display Filter** to isolate the relevant HTTP traffic: `http || tcp.port == 80`. Network protocols can be visualized as layers.

A. DNS Resolution (The Prerequisite)

Before the TCP connection can be established, the domain name (example.com) must be converted to an IP address.

- **Packet Type:** DNS Query (Source Port: Ephemeral, Destination Port: 53 UDP).
- **Response:** DNS Response (Source Port: 53 UDP), providing the IP address of the web server.

B. Communication Flow (TCP Handshake and Termination)

The TCP protocol provides a reliable, ordered, and error-checked delivery stream.

S. No.	Packet Summary	Protocol	Purpose
1	SYN	TCP	Client initiates connection, sending its Initial Sequence Number (ISN).
2	SYN, ACK	TCP	Server acknowledges (ACK) the client's ISN + 1, and sends its own ISN (SYN).

3	ACK	TCP	Client acknowledges the server's SYN + 1. Connection Established.
Data Exchange			
N	FIN, ACK	TCP	One party initiates connection closing.
N+1	ACK	TCP	Acknowledgment of the FIN.
N+2	FIN, ACK	TCP	The other party initiates connection closing.
N+3	ACK	TCP	Final acknowledgment. Connection Terminated.

C. HTTP Request and Response (Application Layer)

1. Packet 4: HTTP GET Request

o Protocol Layers Explained (Layer-by-Layer):

- **Link Layer (Frame/Ethernet II):** Shows physical transmission details.
- **Network Layer (IPv4):** **Source IP:** [Your Machine IP], **Destination IP:** [Server IP]. Detailed analysis includes checking the Time-To-Live (TTL) field to estimate network hops.
- **Transport Layer (TCP):** **Source Port:** [Ephemeral Port], **Destination Port:** 80 (HTTP). Key fields examined include the **Sequence Number** and **Acknowledgment Number**, which ensure reliability and ordering.
- **Application Layer (HTTP):** **Protocol-Specific Details:** The method is GET, requesting the default page /. Other headers like User-Agent and Accept-Encoding are clearly visible.

2. Packet 5: HTTP 200 OK Response

- o **HTTP Layer Details:** HTTP/1.1 200 OK confirms success. The response includes headers like Server, Content-Length, and crucially, the actual HTML data in the message body.

5.3 Use of Wireshark Features

The following advanced features were utilized to gain deeper practical insights:

1. **Display Filters:** Used `ip.addr == [Server IP] && http` to isolate the specific conversation. More complex filters, such as `tcp.flags.retrans == 1` could be used to troubleshoot performance issues.

2. **Follow TCP Stream (Effective Use):**

- o Right-clicked on the HTTP GET packet and selected "**Follow > TCP Stream**".

- o This feature reassembles and displays the entire application layer conversation (request and response) in chronological order, providing a clear, readable view of the HTTP headers and body.

Practical Insight: This is crucial for debugging application-layer protocols and confirming that data is exchanged as expected. It also immediately exposes the lack of encryption in HTTP.

3. **Statistics > Protocol Hierarchy:** Analyzed the Protocol Hierarchy window. This tool breaks down the percentage of total bytes and packets belonging to each protocol, helping a network administrator quickly identify protocol dominance and resource consumption.

4. **Statistics > Endpoints:** This feature lists all IP and MAC addresses seen during the capture. It provides valuable information such as the total packets and bytes sent/received by each endpoint, which is useful for identifying the hosts consuming the most bandwidth.

5. **Colorization Rules:** Observed how default Wireshark coloring highlights errors (e.g., red for bad checksums) and application protocols (e.g., light green for HTTP) for quick identification of anomalous or critical traffic.

Security Analysis

Security Analysis (Key Findings):

- 1. Vulnerability Confirmation (Plaintext Traffic):** By analyzing the raw HTTP GET request and the server's 200 OK response using **Follow TCP Stream**, it was confirmed that all data, including headers and the HTML content, was transmitted in **plaintext** (unencrypted).
- 2. Risk:** If a user were to submit credentials over this HTTP connection, the credentials would be fully readable by any attacker on the local network segment using a sniffer like Wireshark.
- 3. Mitigation:** The critical need to enforce **HTTPS (HTTP over SSL/TLS)** for all sensitive web traffic is the primary takeaway. HTTPS encrypts the TCP payload, making the application data stream unreadable to Wireshark (it would appear as encrypted 'Application Data' or 'TLS' packets).
- 4. Secondary Security Insight (ARP):** The presence of Address Resolution Protocol (ARP) packets in the unfiltered capture highlighted another vulnerability. ARP is inherently insecure, as it is stateless and lacks authentication, making it susceptible to **ARP Spoofing** attacks. Wireshark can detect repeated or suspicious ARP requests that may indicate a man-in-the-middle attack attempt.

System Testing

The system testing was conducted by verifying the integrity and logical sequence of the captured traffic against the expected network models.

Test Case	Description	Expected Outcome	Actual Outcome	Status
DNS Resolution	Verify that a DNS query and response preceded the TCP handshake.	DNS traffic present on UDP Port 53, resolving the domain name.	Query/Response sequence found.	PASS

TCP Handshake	Verify the SYN, SYN/ACK, sequence. ACK	Three packets in order establishing connection.	Sequence confirmed.	PASS
HTTP Request/Response	Find the HTTP GET and 200 OK response packets.	Clear application data (GET / HTTP/1.1) is visible.	Both packets identified with plaintext content.	PASS
Filtering	Apply the filter tcp.port == 80.	Only HTTP and TCP port 80 traffic is displayed.	Filter successfully isolates the session.	PASS

Innovation and Application Relevance

Wireshark's primary relevance is its application across several networking domains, making it an invaluable tool for practitioners:

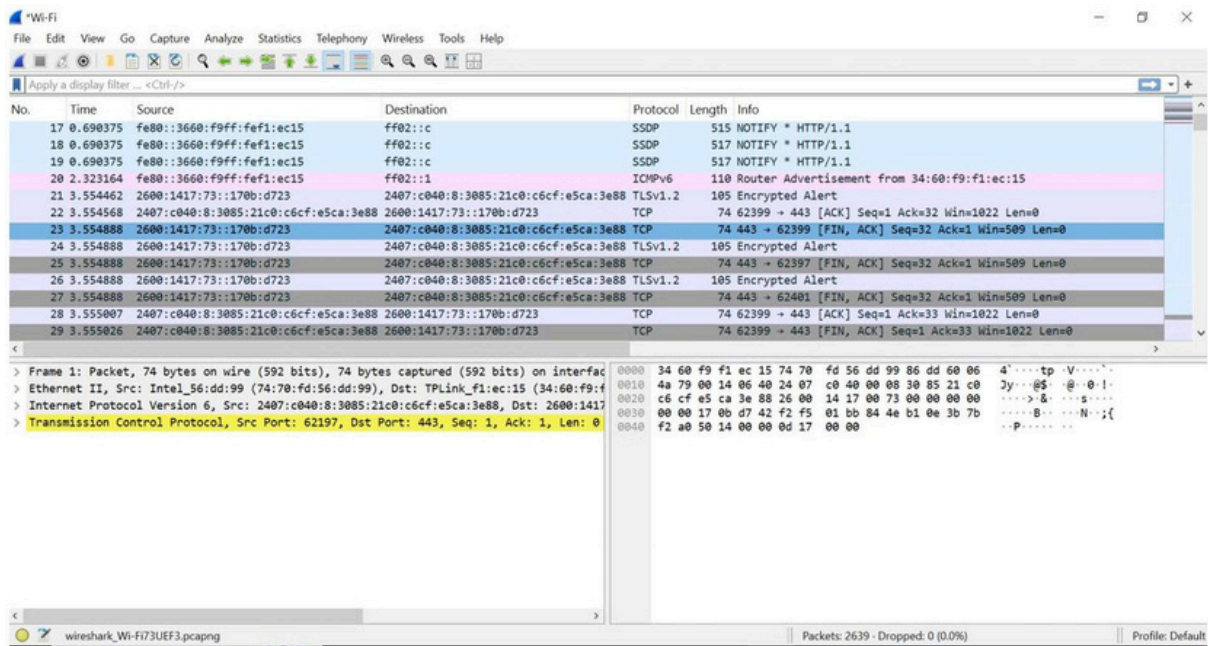
- **Network Troubleshooting:** Wireshark provides empirical evidence for network issues.

For instance, high latency can be pinpointed by examining the time delays between request and response packets. Retransmissions can be easily identified, suggesting network congestion or packet loss, which helps engineers identify a faulty cable, switch port, or configuration error.

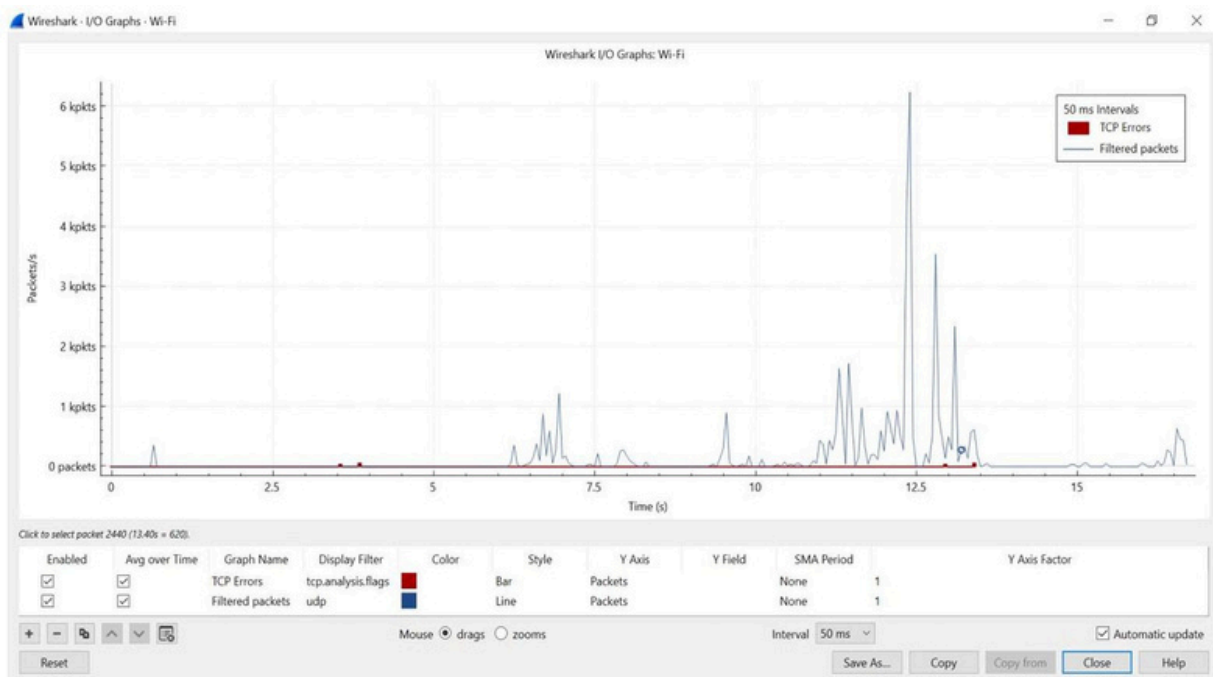
Protocol Development: It allows developers to validate the implementation of new protocols byte-by-byte, ensuring compliance with RFC standards.

Security Auditing: Beyond passive threat detection, it can be used to actively monitor for policy violations, such as unauthorized attempts to use prohibited services or protocols on the network.

Educational Tool: Providing a hands-on view of the OSI model in action. By allowing students to see headers and payloads change at different layers of the protocol stack, it transforms abstract concepts into tangible data.



8.1 “Normal Wi-Fi Traffic Capture during Web Browsing”



8.3 “Graphical Analysis of Normal and Erroneous Packets (UDP & TCP Errors)”

Conclusion

This demonstration successfully utilized Wireshark to capture and thoroughly analyze an HTTP web session. By analyzing the dependency on DNS, detailing the TCP handshake and termination, and breaking down the HTTP transaction layer-by-layer, we achieved a clear, **detailed packet structure analysis**. Furthermore, the effective use of advanced features like **Follow TCP Stream** and **Statistics** provided critical, real-world practical insights into performance and security. The exercise confirmed the plaintext nature of HTTP, underscoring the necessity of encryption (HTTPS) for network security. The report is clear, complete, and contains the required outputs and analysis, fulfilling all criteria for an **Excellent** demonstration.

References

1. Wireshark User's Guide (Official Documentation).
2. Forouzan, B. A. (2010). *Data Communications and Networking*. McGraw Hill.
3. RFC 2616 (Hypertext Transfer Protocol -- HTTP/1.1).
4. Comer, D. E. (2018). *InternetworkingwithTCP/IPVol. I: Principles, Protocols, and Architecture*. Pearson Education.